BookNook – CSCI426 Project Report

Title Page

Department of Computer Science and Information Technology

CSCI426: Advanced Web Programming

Project Phase 1

BookNook: An Online Bookstore Built with ReactJS and Bootstrap

Student name	Elya Semaan
Student ID	92430097
Instructor	Ihad Hassoun
Submission date	November 2026
GitHub repository	https://github.com/92430097-wq/booknook
Live website	https://booknook-smoky.vercel.app

Abstract

BookNook is a responsive frontend web application for an online bookstore. Users can browse books, search by title or author, filter by genre, sort by price, view the details of each book, and purchase books through a shopping cart and checkout. The application is built with ReactJS and Bootstrap 5, uses React Router for navigation between six pages, and shares the cart between pages using the React Context API. The cart is saved in the browser with localStorage, so it is kept after a page refresh. The project is managed with Git and GitHub and deployed on Vercel.

Introduction

Finding a specific book in a physical store can take time, and it is hard to compare prices or see what else is available in the same genre. BookNook solves this with a simple online store where readers can find books quickly and prepare an order from any device.

The main goals of the project are:

- Build a frontend web application with ReactJS and Bootstrap.
- Let users browse, search and purchase books.

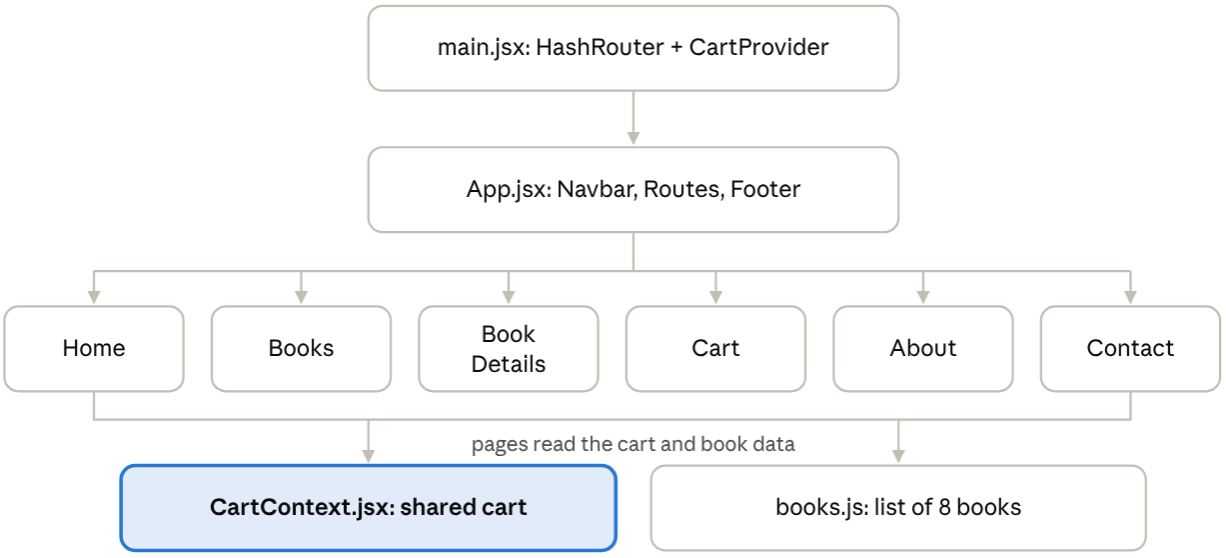
- Apply UI/UX principles: a clear layout, consistent colors, and easy navigation.
- Make the website responsive so it works on desktop, tablet, and mobile.
- Use Git for version control and deploy the website online.

System Design

The application is a single-page React app with six pages, reusable components, and one shared cart.

Component structure

Six pages share one cart through React Context



component structure · 6 pages, 1 shared cart

`main.jsx` wraps the app in the router and the cart provider, `App.jsx` holds the navbar, footer and routes, and every page can read the cart and the book list.

Pages

Page	Route	What it does
Home	`/`	Welcome banner, store features and featured books
Books	`/books`	All books with search by title or author, genre filter and price sort

Page	Route	What it does
Book Details (dynamic)	`/books/:id`	Shows one book based on the id in the URL, plus related books
Cart	`/cart`	Lists cart items, changes quantities, shows the total, checkout
About	`/about`	Mission, books and team
Contact	`/contact`	Contact information and a contact form

Folder structure

```
src/
  components/   Navbar, Footer, BookCard, BookCover
  context/      CartContext (cart logic)
  data/         books.js (book list)
  pages/        Home, Books, BookDetails, Cart, About, Contact
  App.jsx       Routes
  main.jsx      Entry point
```

Purchase flow

1. The user finds a book by browsing, searching or filtering.
2. The user clicks **Add to Cart** on a book card or the details page.
3. `addToCart` in CartContext adds the book or increases its quantity, and the navbar badge updates.
4. On the Cart page the user changes quantities, sees the total and clicks **Checkout**.
5. The cart is saved to localStorage so it stays after a refresh.

UI/UX and responsive design

- A calm green and cream color theme that suits a bookstore, used the same way on every page.
- Book covers drawn with CSS, so every book has a clean, consistent cover.
- A sticky navbar that collapses into a menu button on mobile.
- Bootstrap grid: 2 books per row on mobile, 3 on tablet, 4 on desktop.
- Clear feedback for the user: "Added to your cart!", an order confirmation, and an empty-cart message.

Technologies Used

Technology	Use in the project
ReactJS 18	Builds the user interface from reusable components
Vite	Runs the development server and builds the project
React Router DOM 6	Navigation between pages and the dynamic route `/books/:id`
Bootstrap 5	Grid system, navbar, cards, forms, buttons and responsive layout
React Context API	Shares the cart between all pages
localStorage	Saves the cart in the browser
CSS	Custom green theme, CSS book covers and hover effects
Git and GitHub	Version control and source code hosting
Vercel	Hosting the live website
Visual Studio Code	Code editor

Code Snippets

1. Routing (App.jsx)

Each page has its own route. The route `/books/:id` is the dynamic page.

```
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/about" element={<About />} />
  <Route path="/books" element={<Books />} />
  <Route path="/books/:id" element={<BookDetails />} />
  <Route path="/cart" element={<Cart />} />
  <Route path="/contact" element={<Contact />} />
</Routes>
```

2. Search, filter and sort (Books.jsx)

Books are filtered by title or author and by genre, then sorted by price if the user chooses it.

```
let filtered = books.filter((b) => {
  const text = search.toLowerCase();
  const matchSearch =
    b.title.toLowerCase().includes(text) ||
    b.author.toLowerCase().includes(text);
  const matchGenre = genre === "All" || b.genre === genre;
  return matchSearch && matchGenre;
});

if (sort === "low") filtered = [...filtered].sort((a, b) => a.price -
b.price);
if (sort === "high") filtered = [...filtered].sort((a, b) => b.price -
a.price);
```

3. Add to cart (CartContext.jsx)

If the book is already in the cart, its quantity goes up by one. If not, it is added with a quantity of 1.

```
function addToCart(product) {
  const exists = cart.find((item) => item.id === product.id);
  if (exists) {
    setCart(
      cart.map((item) =>
        item.id === product.id ? { ...item, qty: item.qty + 1 } : item
      )
    );
  } else {
    setCart([...cart, { ...product, qty: 1 }]);
  }
}
```

4. Saving the cart (CartContext.jsx)

```
const [cart, setCart] = useState(() => {
  const saved = localStorage.getItem("cart");
  return saved ? JSON.parse(saved) : [];
});

useEffect(() => {
  localStorage.setItem("cart", JSON.stringify(cart));
}, [cart]);
```

5. Dynamic book page (BookDetails.jsx)

The book id is read from the URL, then the matching book and related books from the same genre are found.

```
const { id } = useParams();
const book = books.find((b) => b.id === Number(id));
const related = books.filter((b) => b.genre === book.genre && b.id !==
book.id);
```

6. Reusable CSS book cover (BookCover.jsx)

Each cover is drawn with CSS using the book's color, so no images are needed.

```
<div className="book-cover" style={{ backgroundColor: book.color }}>
  <div className="book-cover-title">{book.title}</div>
  <div className="book-cover-line"></div>
  <div className="book-cover-author">{book.author}</div>
</div>
```

Screenshots

The figures below show the main pages of the running website on desktop and on a phone.

Figure 1: Home page

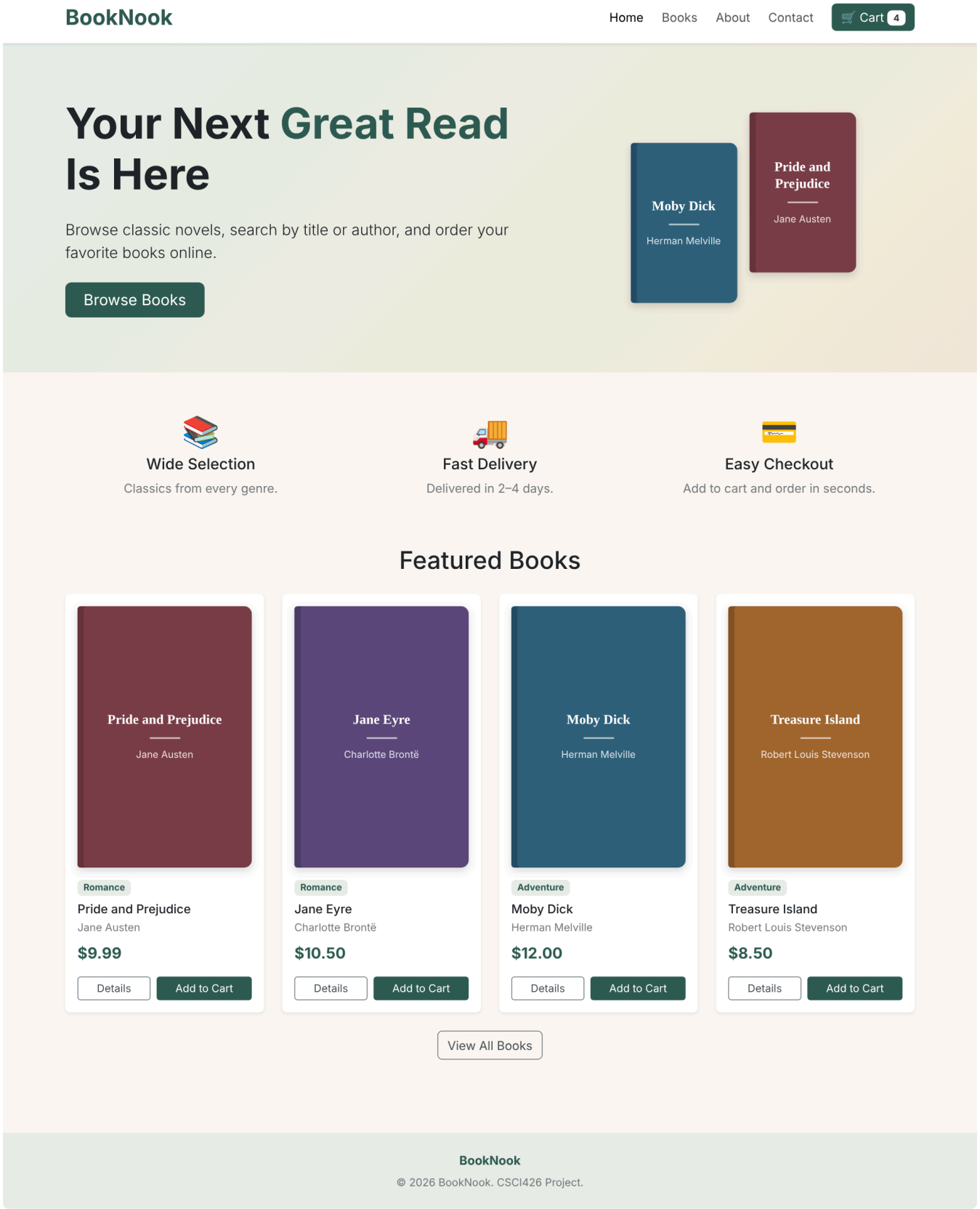


Figure 2: Browse Books page filtered by the Horror genre

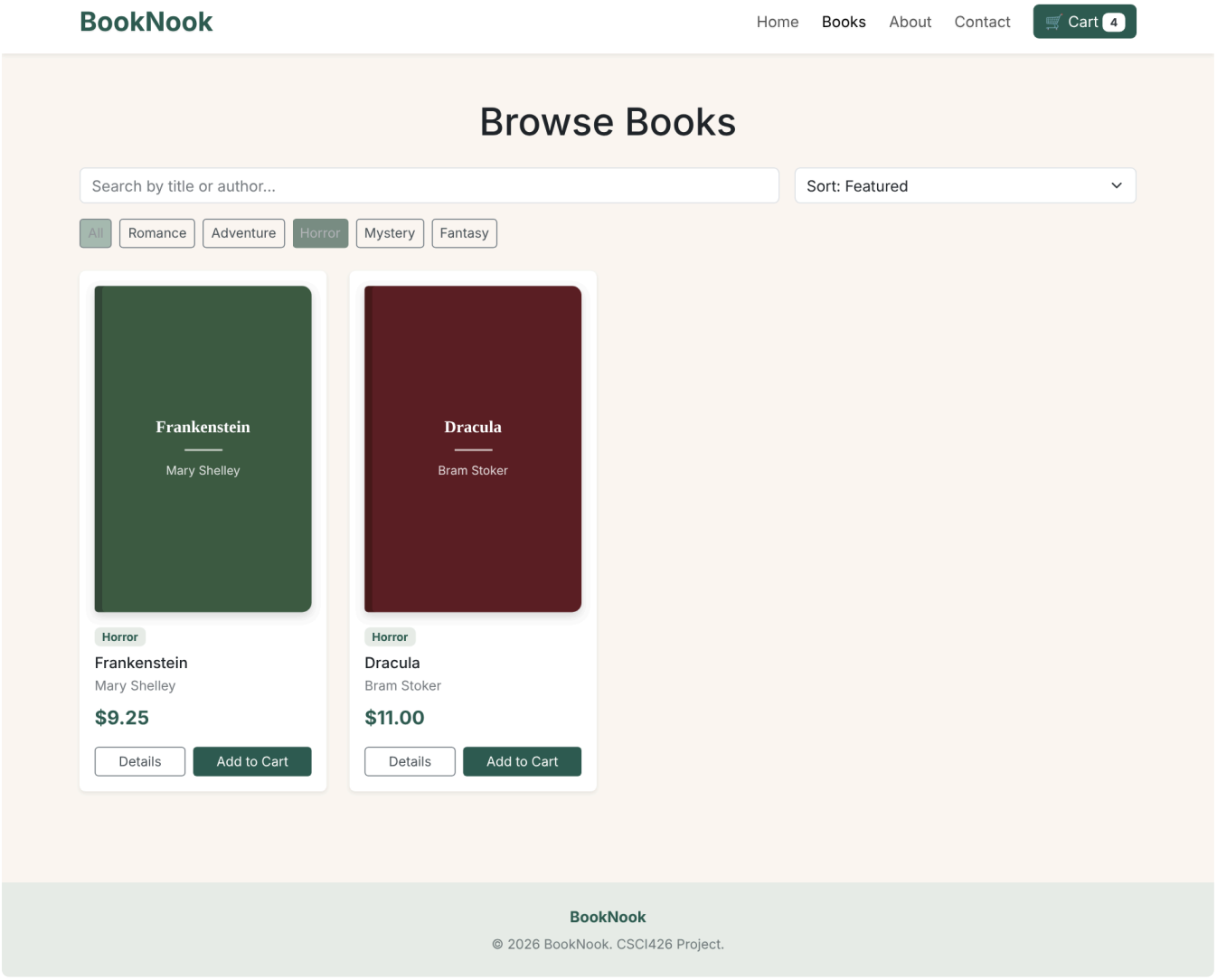


Figure 3: Book details page after adding the book to the cart

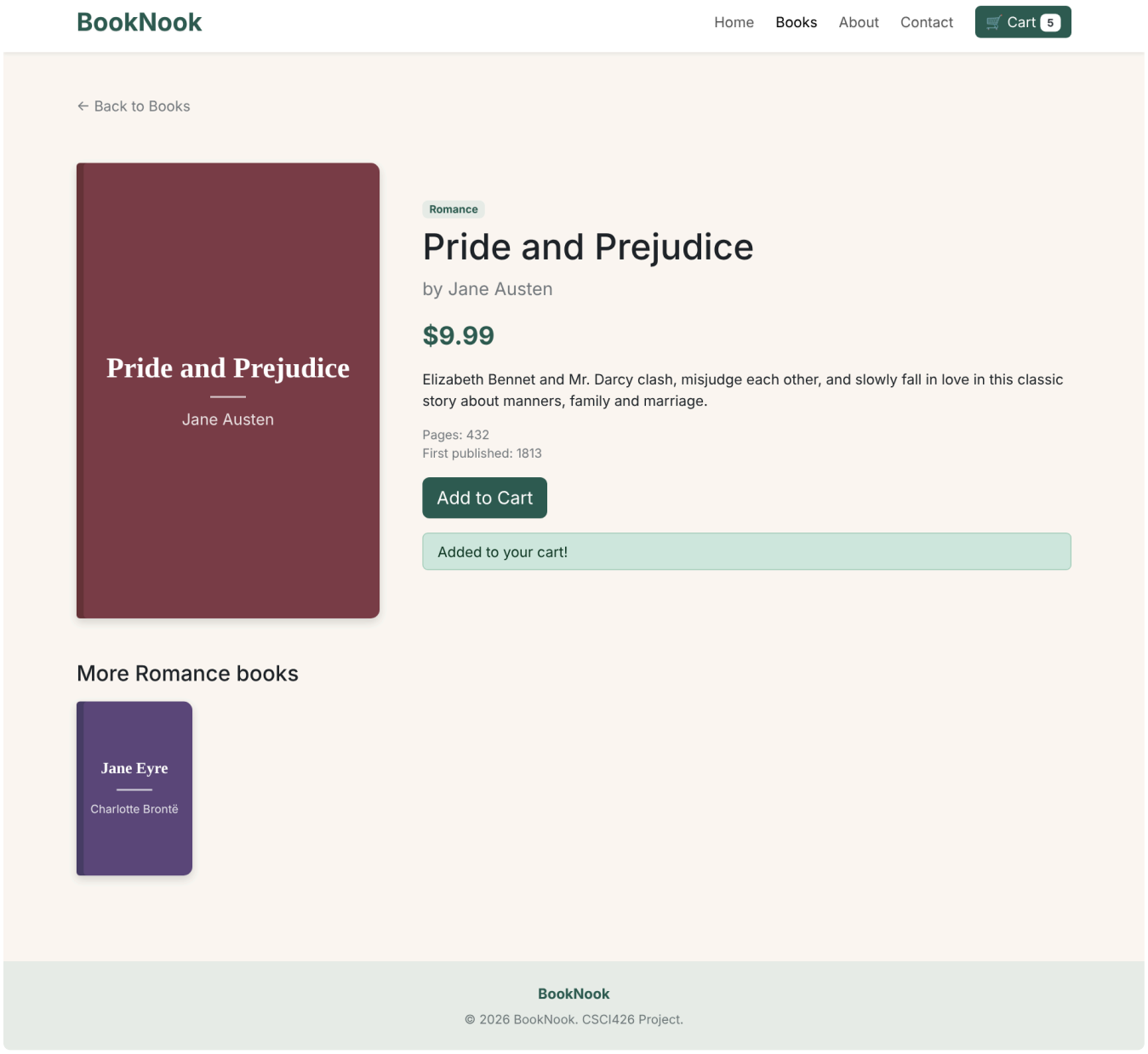


Figure 4: Cart page

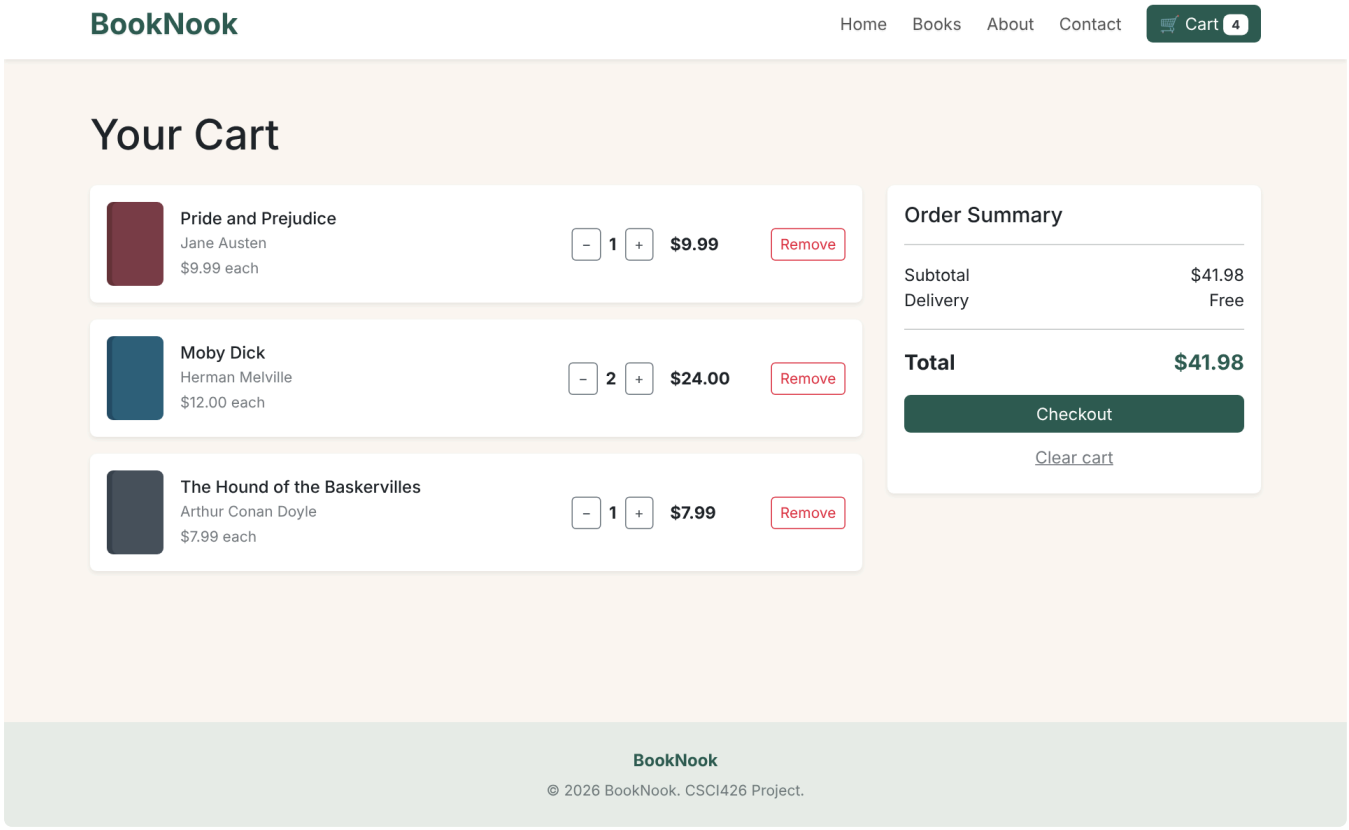


Figure 5: Contact page

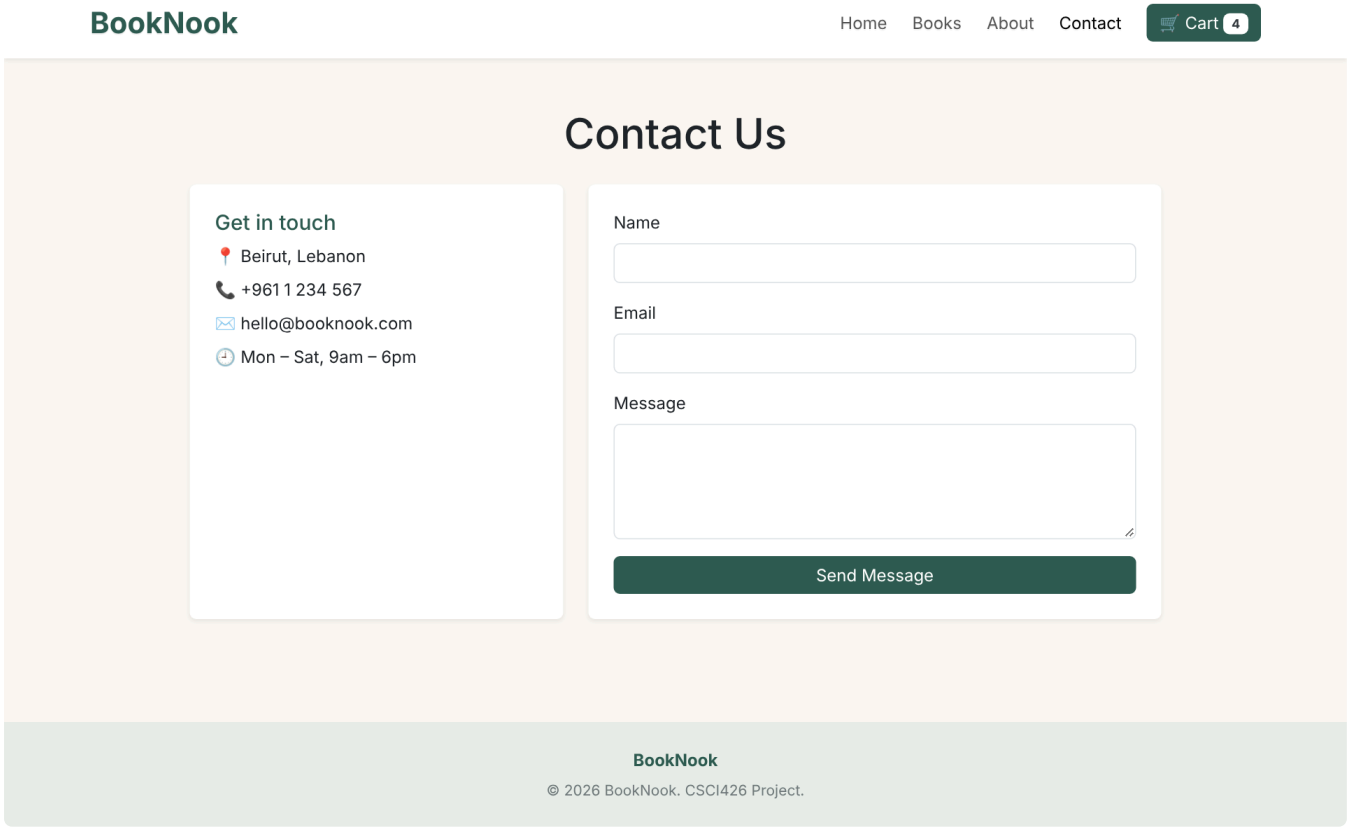
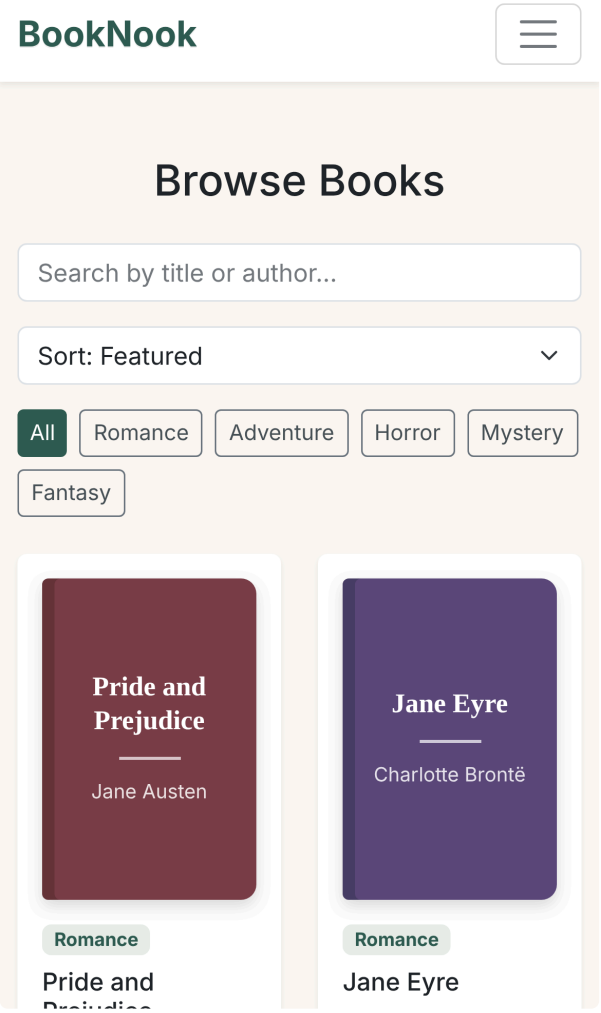


Figure 6: Mobile view of the Browse Books page



Version Control and Deployment

The project was tracked with Git from the start, and each feature was saved in its own commit (for example: project setup, navbar and routing, book list, search and filter, cart, contact page, README). The code is hosted on GitHub.

The website is deployed on Vercel. The Vercel project is connected to the GitHub repository, so every push to the main branch rebuilds and redeploys the site automatically (build command npm run build, output folder dist).

The project uses `HashRouter`, so refreshing any page works correctly on Vercel.

- GitHub repository: https://github.com/92430097-wq/booknook
- Live website: https://booknook-smoky.vercel.app

Setup instructions: install Node.js, clone the repository, run `npm install`, then run `npm run dev` and open the link shown in the terminal.

Conclusion

BookNook meets the Phase 1 requirements: it is a ReactJS and Bootstrap frontend with six pages, including a dynamic book page, search, genre filter, price sort, a working cart and checkout, and a responsive design for desktop and mobile. It is managed with Git and deployed online. In Phase 2, the store can be connected to a backend and a database to add user accounts, real payments, and order tracking.

Contribution Statement

This project was completed individually by Elya Semaan (ID 92430097), who did the design, development, testing, deployment, and documentation.